FUZZING

Fuzzing is an automated software testing technique that involves providing unexpected or random data as input to a computer program. The goal is to uncover bugs, crashes, or memory safety violations that might otherwise go unnoticed.

Fuzzing or fuzz testing is an automated software testing technique that involves providing invalid, unexpected, or random data as inputs to a computer program. The program is then monitored for exceptions such as crashes, or failing built-in code assertions or for finding potential memory leaks

Grammar-based Fuzzing :

• Idea: Implement the file format / protocol inside your fuzzer
• Example: “at offset 4 is an unsigned dword, at offset 10 is a checksum, at offset 14 is a nullterminated string, at offset 20 a type field, …”
• Covers everything which you defined (but maybe forgets corner cases which you didn’t think of)
• After the (long) initial work, the fuzzer covers lots of corner cases very fast
• Examples: Peach, Domato, Boofuzz, Sulley, Spike, …

Feedback-based Fuzzing:

• Let the fuzzer learn the file format itself ➔ No initial work required (fast results)
• However, learning the format can take a long time and maybe some checks could not be learned by the fuzzer
• ➔ If we build better feedback-based fuzzers it has no (or just small) drawbacks
• Examples: AFL, WinAFL, LibFuzzer, Honggfuzz, …
• Side note: Use this technique for binary inputs. For “interactive” inputs (e.g.: JavaScript / DOM in browser or shell-like software which listens on a port) this technique is only partially useful

```
if (input_buffer[2] == 0x55) {
    printf("0x55 case\n");
}
else {
    printf("Default case\n");
}
if (input_buffer[3] == 0xaa) {
    if (input_buffer[4] == 0xff) {
        if (input_buffer[5] == 0xcc) {
            printf("Vulnerability triggered!\n");
        }
    }
}
```

Fuzzer Input:

Input 1: 00 00 00 00 00 00

Fuzzer queue:

Input 1: 00 00 00 00 00 00

libfuzzer, as the name suggests is used for fuzzing libraries mainly. (Libraries are .so files in linux and .dll in Windows; they contain code which can be resused by other programs)

Set up the environment, write a simple yet effective "fuzzing harness," and use powerful tools
like **libFuzzer** and **AddressSanitizer (ASan)** to find crashes, which might be a bug.

Steps:

1. Install prerequisites:

```
apt update -y && apt install clang cmake make git libtool autoconf automake -y
```

2. This will be the directory structure:

```
~/fuzzing-json-c/
├── json-c/              # Cloned json-c source code
├── fuzz/                # Custom fuzzing harness and corpus
|    ├── fuzz_json_tokener.cpp
|    ├── build.sh        # Helper script to build fuzzer
|    └── corpus/         # Seed inputs for fuzzing
```

3. Compiling source code with

```
mkdir fuzzing-json-c && cd fuzzing-json-c
git clone https://github.com/json-c/json-c
cd json-c
mkdir build && cd build
CC=clang CXX=clang++ CFLAGS="-fsanitize=address,fuzzer-no-link -g" CXXFLAGS="-fsanitize=address,fuzzer-no-link -g" cmake ..
make -j$(nproc)
```

Step-by-Step Breakdown

a. Set the Compiler to `clang` (Required for Fuzzing)

```
CC=clang CXX=clang++
```

- This tells your system: "*Use the `clang` compiler instead of the default one.*"

b. Add Safety Checks (`AddressSanitizer`)

```
CFLAGS="-fsanitize=address,fuzzer-no-link -g"
```

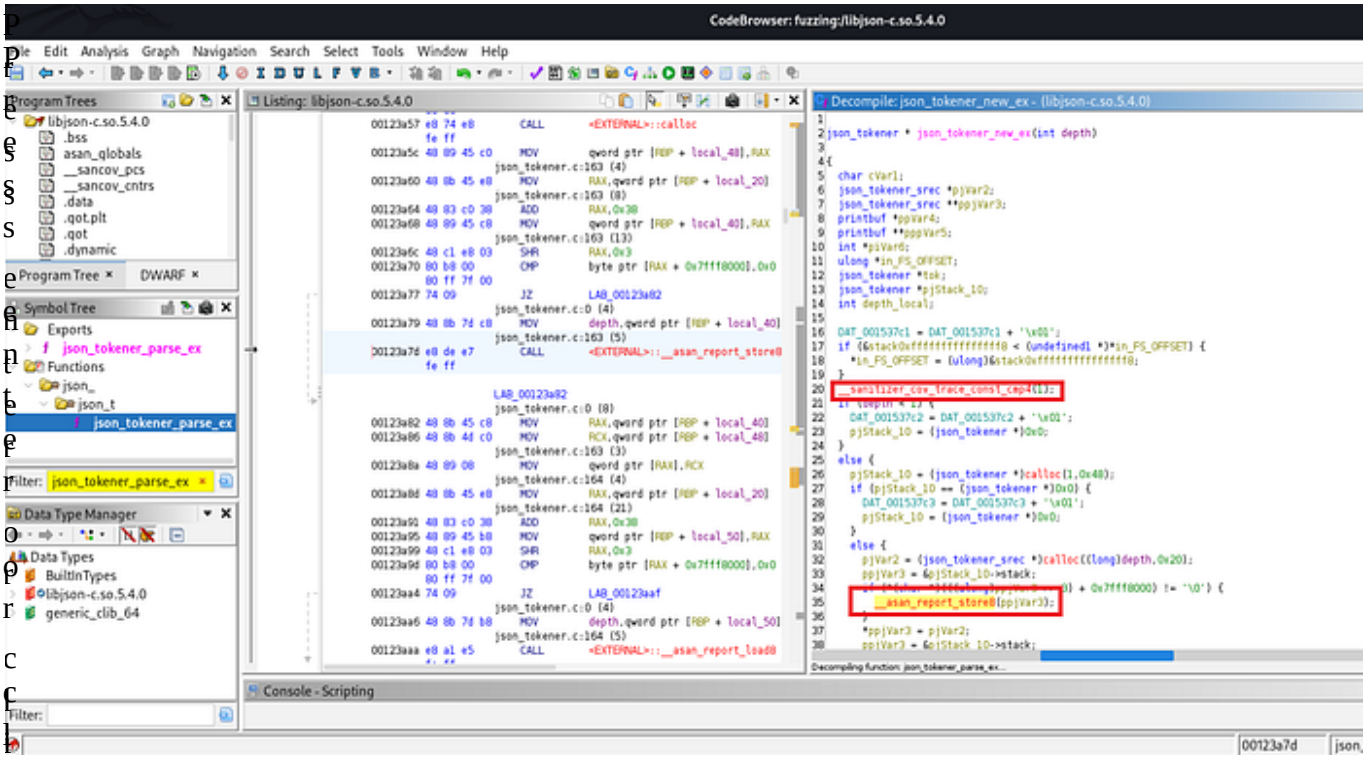
- `-fsanitize=address` → Detects memory bugs (crashes, leaks).
- `fuzzer-no-link` → Prepares for fuzzing but doesn't enable it yet.
- `-g` → Adds debug info (so crashes make sense).

c. Run CMake

```
# .. Tell CMake: "The project files are up one level"
cmake ..
```

- This generates build files with the above settings.

So clearly new .so file is different than regular.



Feature	Regular Version	Instrumented Version (fuzzing)
Memory safety checks	❌	✅ `__asan_report_*`
Coverage feedback hooks	❌	✅ `__sanitizer_cov_trace_*`
Extra counters/statistics	❌	✅ `DAT_001537cX++`
Debug/disassembly symbols	Friendly names	Mangled/generated names
Fuzzer-aware compilation	❌	✅ (`-fsanitize=fuzzer,address`)

4. Writing **Fuzzing Harness**:

It's a small function that takes the fuzzer-generated data and passes it to the target function. For `libFuzzer`, this entry point must be named `LLVMFuzzerTestOneInput`.

/fuzzing-json-c/fuzz/fuzz_json_tokener.cpp

```
// ~/fuzzing-json-c/fuzz/fuzz_json_tokener.cpp
```

```c
#include "../json-c/json_tokener.h" //../json-c/json_tokener.h: This includes the header file for the specific json-c function we want to test (json_tokener_parse_ex)
#include <stdint.h>
#include <stdlib.h>
#include <string.h>
extern "C" int LLVMFuzzerTestOneInput(const uint8_t *data, size_t size) {
    //Input preparation
    char *input = (char *)malloc(size + 1);
    if (!input) return 0;
    memcpy(input, data, size);
    input[size] = '\0';

    //Calling the Target Function
    struct json_tokener *tok = json_tokener_new();
    struct json_object *obj = json_tokener_parse_ex(tok, input, size);
    if (obj) json_object_put(obj);
    json_tokener_free(tok);
    free(input);
    return 0;
}
```

Step-by-Step Breakdown of “**Input Prep**” section:

The `json_tokener_parse_ex` function expects a standard C string, which must end with a null terminator (`\0`). The raw `data` from the fuzzer doesn't have one.

a.`malloc(size + 1)`: Allocates enough memory for the fuzzer's data plus one extra byte for the null terminator.

b.`memcpy(input, data, size)`: Copies the random bytes from `data` into the newly allocated `input` buffer.

c.`input[size] = '\0'`: Adds the crucial null terminator at the end to make it a valid C string.

Step-by-Step Breakdown of "**Calling Function**" section:

a.`json_tokener_new()`: Creates a new JSON tokener object required by the parsing function.

b.`json_tokener_parse_ex(...)`: This is the **target function call**. It attempts to parse the fuzzer-generated `input` string. If the input is malformed in a way the developers didn't anticipate, this is where a crash or memory error will likely occur.

c.`if (obj) json_object_put(obj)`: **Crucial cleanup step.** If the parser successfully creates a JSON object (`obj`), this line frees the memory associated with it. This prevents the fuzzer from reporting memory leaks.

d.`json_tokener_free(tok)` and `free(input)`: Frees the memory allocated for the tokenizer and the input string, ensuring no memory is leaked during the fuzzing loop.

5. The next step is to **Compile the Fuzzer:**

Create build.sh to compile.

```
#!/bin/bash
clang++ -fsanitize=address,fuzzer fuzz_json_tokener.cpp \
    -I../json-c \
    -I../json-c/build \
    ../json-c/build/libjson-c.a \
```

```
-o fuzz_json_tokener
```

After running chmod+x build.sh and ./build.sh, We will have **fuzz_json_tokener** binary

```
┌──(root㉿vm)-[~/fuzzing-json-c/fuzz]
└─# ls -al
total 16
drwxr-xr-x 2 root root 4096 Jul 26 13:48 .
drwxr-xr-x 4 root root 4096 Jul 26 13:46 ..
-rwxr-xr-x 1 root root  126 Jul 26 14:23 build.sh
-rw-r--r-- 1 root root  565 Jul 26 14:22 fuzz_json_tokener.cpp

┌──(root㉿vm)-[~/fuzzing-json-c/fuzz]
└─# subl build.sh

┌──(root㉿vm)-[~/fuzzing-json-c/fuzz]
└─# ./build.sh

┌──(root㉿vm)-[~/fuzzing-json-c/fuzz]
└─# ls -al
total 2420
drwxr-xr-x 2 root root    4096 Jul 26 14:23 .
drwxr-xr-x 4 root root    4096 Jul 26 13:46 ..
-rwxr-xr-x 1 root root     164 Jul 26 14:23 build.sh
-rwxr-xr-x 1 root root 2457832 Jul 26 14:23 fuzz_json_tokener
-rw-r--r-- 1 root root     565 Jul 26 14:22 fuzz_json_tokener.cpp
```

6. Crating sample test files, also known as corpus inside fuzz dir:

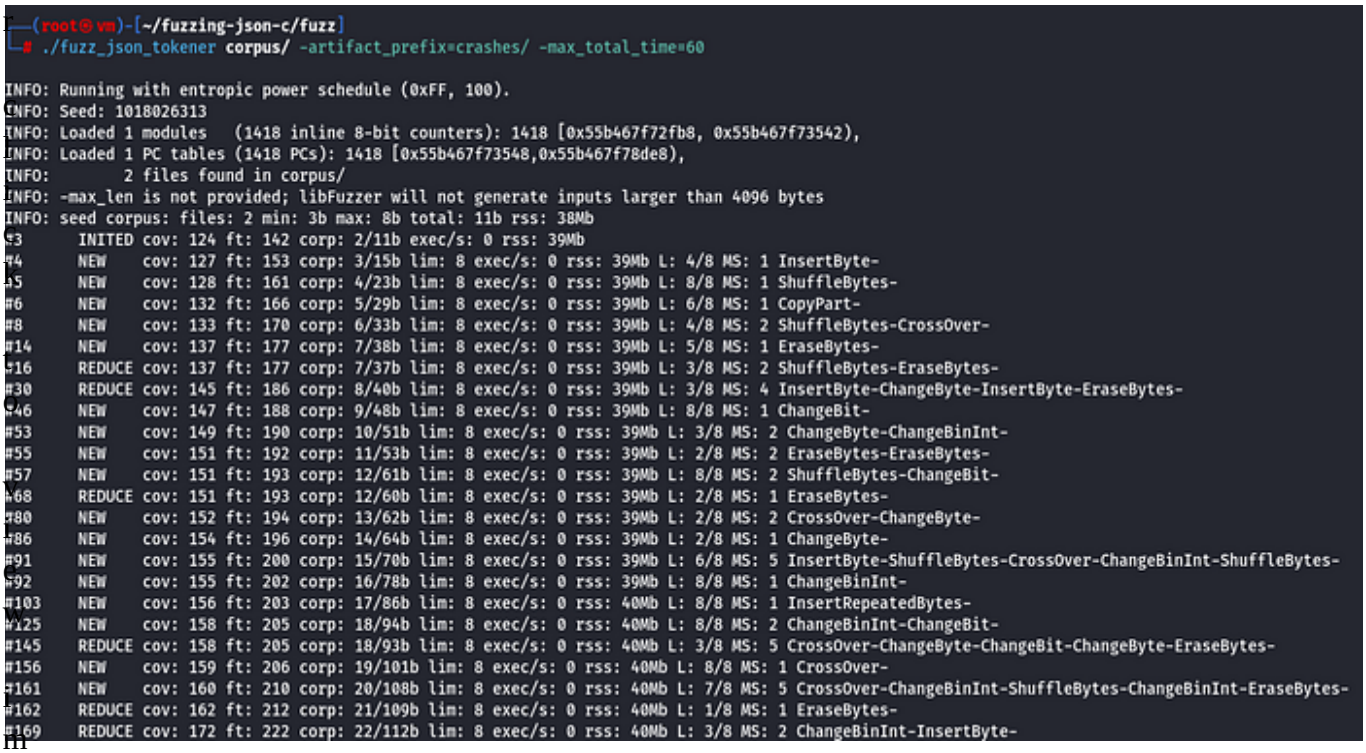
```
mkdir -p corpus
echo '{}' > corpus/seed1.json
echo '{"a":1}' > corpus/seed2.json
```

If you run the fuzzer without providing a corpus directory, it will start from an “empty” or random state. It begins by generating completely random byte sequences and feeding them to the target program.

For a structured format like JSON, this is inefficient. Most random inputs (e.g., `"\xAB\x12\xCD\x34"`) will be immediately rejected by the parser's initial checks because they don't start with a `{` or `[`. The fuzzer will have to spend most of its time and CPU cycles just to "learn" the basic syntax of JSON before it can start testing the deeper logic.

7. Finally last step, Start Fuzzing now:

```
./fuzz_json_tokener corpus/ -artifact_prefix=crashes/ -max_total_time=60
```



```
┌──(root㉿vm)-[~/fuzzing-json-c/fuzz]
└─# ./fuzz_json_tokener corpus/ -artifact_prefix=crashes/ -max_total_time=60

INFO: Running with entropic power schedule (0xFF, 100).
INFO: Seed: 1018026313
INFO: Loaded 1 modules   (1418 inline 8-bit counters): 1418 [0x55b467f72fb8, 0x55b467f73542),
INFO: Loaded 1 PC tables (1418 PCs): 1418 [0x55b467f73548,0x55b467f78de8),
INFO:        2 files found in corpus/
INFO: -max_len is not provided; libFuzzer will not generate inputs larger than 4096 bytes
INFO: seed corpus: files: 2 min: 3b max: 8b total: 11b rss: 38Mb
#3      INITED cov: 124 ft: 142 corp: 2/11b exec/s: 0 rss: 39Mb
#4      NEW    cov: 127 ft: 153 corp: 3/15b lim: 8 exec/s: 0 rss: 39Mb L: 4/8 MS: 1 InsertByte-
#5      NEW    cov: 128 ft: 161 corp: 4/23b lim: 8 exec/s: 0 rss: 39Mb L: 8/8 MS: 1 ShuffleBytes-
#6      NEW    cov: 132 ft: 166 corp: 5/29b lim: 8 exec/s: 0 rss: 39Mb L: 6/8 MS: 1 CopyPart-
#8      NEW    cov: 133 ft: 170 corp: 6/33b lim: 8 exec/s: 0 rss: 39Mb L: 4/8 MS: 2 ShuffleBytes-CrossOver-
#14     NEW    cov: 137 ft: 177 corp: 7/38b lim: 8 exec/s: 0 rss: 39Mb L: 5/8 MS: 1 EraseBytes-
#16     REDUCE cov: 137 ft: 177 corp: 7/37b lim: 8 exec/s: 0 rss: 39Mb L: 3/8 MS: 2 ShuffleBytes-EraseBytes-
#30     REDUCE cov: 145 ft: 186 corp: 8/40b lim: 8 exec/s: 0 rss: 39Mb L: 3/8 MS: 4 InsertByte-ChangeByte-InsertByte-EraseBytes-
#46     NEW    cov: 147 ft: 188 corp: 9/48b lim: 8 exec/s: 0 rss: 39Mb L: 8/8 MS: 1 ChangeBit-
#53     NEW    cov: 149 ft: 190 corp: 10/51b lim: 8 exec/s: 0 rss: 39Mb L: 3/8 MS: 2 ChangeByte-ChangeBinInt-
#55     NEW    cov: 151 ft: 192 corp: 11/53b lim: 8 exec/s: 0 rss: 39Mb L: 2/8 MS: 2 EraseBytes-EraseBytes-
#57     NEW    cov: 151 ft: 193 corp: 12/61b lim: 8 exec/s: 0 rss: 39Mb L: 8/8 MS: 2 ShuffleBytes-ChangeBit-
#68     REDUCE cov: 151 ft: 193 corp: 12/60b lim: 8 exec/s: 0 rss: 39Mb L: 2/8 MS: 1 EraseBytes-
#80     NEW    cov: 152 ft: 194 corp: 13/62b lim: 8 exec/s: 0 rss: 39Mb L: 2/8 MS: 2 CrossOver-ChangeByte-
#86     NEW    cov: 154 ft: 196 corp: 14/64b lim: 8 exec/s: 0 rss: 39Mb L: 2/8 MS: 1 ChangeByte-
#91     NEW    cov: 155 ft: 200 corp: 15/70b lim: 8 exec/s: 0 rss: 39Mb L: 6/8 MS: 5 InsertByte-ShuffleBytes-CrossOver-ChangeBinInt-ShuffleBytes-
#92     NEW    cov: 155 ft: 202 corp: 16/78b lim: 8 exec/s: 0 rss: 39Mb L: 8/8 MS: 1 ChangeBinInt-
#103    NEW    cov: 156 ft: 203 corp: 17/86b lim: 8 exec/s: 0 rss: 40Mb L: 8/8 MS: 1 InsertRepeatedBytes-
#125    NEW    cov: 158 ft: 205 corp: 18/94b lim: 8 exec/s: 0 rss: 40Mb L: 8/8 MS: 2 ChangeBinInt-ChangeBit-
#145    REDUCE cov: 158 ft: 205 corp: 18/93b lim: 8 exec/s: 0 rss: 40Mb L: 3/8 MS: 5 CrossOver-ChangeByte-ChangeBit-ChangeByte-EraseBytes-
#156    NEW    cov: 159 ft: 206 corp: 19/101b lim: 8 exec/s: 0 rss: 40Mb L: 8/8 MS: 1 CrossOver-
#161    NEW    cov: 160 ft: 210 corp: 20/108b lim: 8 exec/s: 0 rss: 40Mb L: 7/8 MS: 5 CrossOver-ChangeBinInt-ShuffleBytes-ChangeBinInt-EraseBytes-
#162    REDUCE cov: 162 ft: 212 corp: 21/109b lim: 8 exec/s: 0 rss: 40Mb L: 1/8 MS: 1 EraseBytes-
#169    REDUCE cov: 172 ft: 222 corp: 22/112b lim: 8 exec/s: 0 rss: 40Mb L: 3/8 MS: 2 ChangeBinInt-InsertByte-
```

So we ran it for 60 seconds..

Preserve memory in full size

```
"\377\377\377\377" # Uses: 1003
"\377\377\377\275\275" # Uses: 831
"\3776" # Uses: 920
"\3772" # Uses: 888
"\361\377\377\377" # Uses: 855
"4," # Uses: 853
"\000\000\000\000" # Uses: 835
"1\000\000\000\000\000\000\000" # Uses: 786
"\377\377\377\000" # Uses: 784
"\377\377\377\377\377\377\377\016" # Uses: 713
"\006\000\000\000\000\000\000\000" # Uses: 715
"\377\377\377\377\377\377\377\377\377\377\377\377\377\377\377\377\377\377\377\377\377\377\377\377\377\177" # Uses: 599
"a\305\377\377\377\377\377\377\177" # Uses: 612
"\377\377\377\377\377\377\377\326\377\377\377\377\377\377\377\377\377\377\377\377\377\377\377\331\377\377\377\377\377\377\377\377\377\377\377\377\377\377\377\377\177" # Uses: 451
"\377\377\377\377\377\377\377\221" # Uses: 577
"\376\377\377\377\377\377\377\377" # Uses: 536
"\377\377\001\267" # Uses: 394
"\377\377\377\377\377\377\377\177" # Uses: 391
"\000\000\000\000\000\000\000\012" # Uses: 410
"\377\377\377\377\377\377\377\377\377\377000000000000000000000000000000" # Uses: 336
"\001\000\000\000\000\000\000\000n" # Uses: 351
"\377\213\377\377\377\377\377\377\377\177" # Uses: 305
"\200\377\377\377\377\377\377\377" # Uses: 267
"fa6" # Uses: 347
"\015\000" # Uses: 233
"\377\377\377\377\377\377\377\221\377\377\377\377\377\377\377\377\377\377\377\377\377\357" # Uses: 182
"\377\377\377\377\377\377\377\377\377\377\377\377\377\377\377\377\377\377\377\377\377\377\377\377\377\377\377" # Uses: 211
"\377\377\377\033" # Uses: 100
"\377\377\377\377\377\377\377\377\377\377\377\377\377\177\377\377\377\377\377\377\377\377\177\377\377\377\377\003\314\377\377\377\377\377\377\177" # Uses: 151
"\000\012" # Uses: 146
"\377\213\377\377\377\377\377\377\377" # Uses: 121
"\011\000\000\000\000\000\000\000" # Uses: 88
"\377\001" # Uses: 60
###### End of recommended dictionary. ######
Done 1826862 runs in 61 second(s)
```

And if there is any crash, you will find that in the **crashes** directory.

```
┌──(root㉿vm)-[~/fuzzing-json-c/fuzz]
└─# ls -al
total 2488
drwxr-xr-x 4 root root    4096 Jul 26 14:26 .
drwxr-xr-x 4 root root    4096 Jul 26 14:25 ..
-rwxr-xr-x 1 root root     164 Jul 26 14:23 build.sh
drwxr-xr-x 2 root root   61440 Jul 26 14:27 corpus
drwxr-xr-x 2 root root    4096 Jul 26 14:26 crashes
-rwxr-xr-x 1 root root 2457832 Jul 26 14:23 fuzz_json_tokener
-rw-r--r-- 1 root root     565 Jul 26 14:22 fuzz_json_tokener.cpp

┌──(root㉿vm)-[~/fuzzing-json-c/fuzz]
└─# ls -al ./crashes
total 8
drwxr-xr-x 2 root root 4096 Jul 26 14:26 .
drwxr-xr-x 4 root root 4096 Jul 26 14:26 ..

┌──(root㉿vm)-[~/fuzzing-json-c/fuzz]
└─#
```

Page size